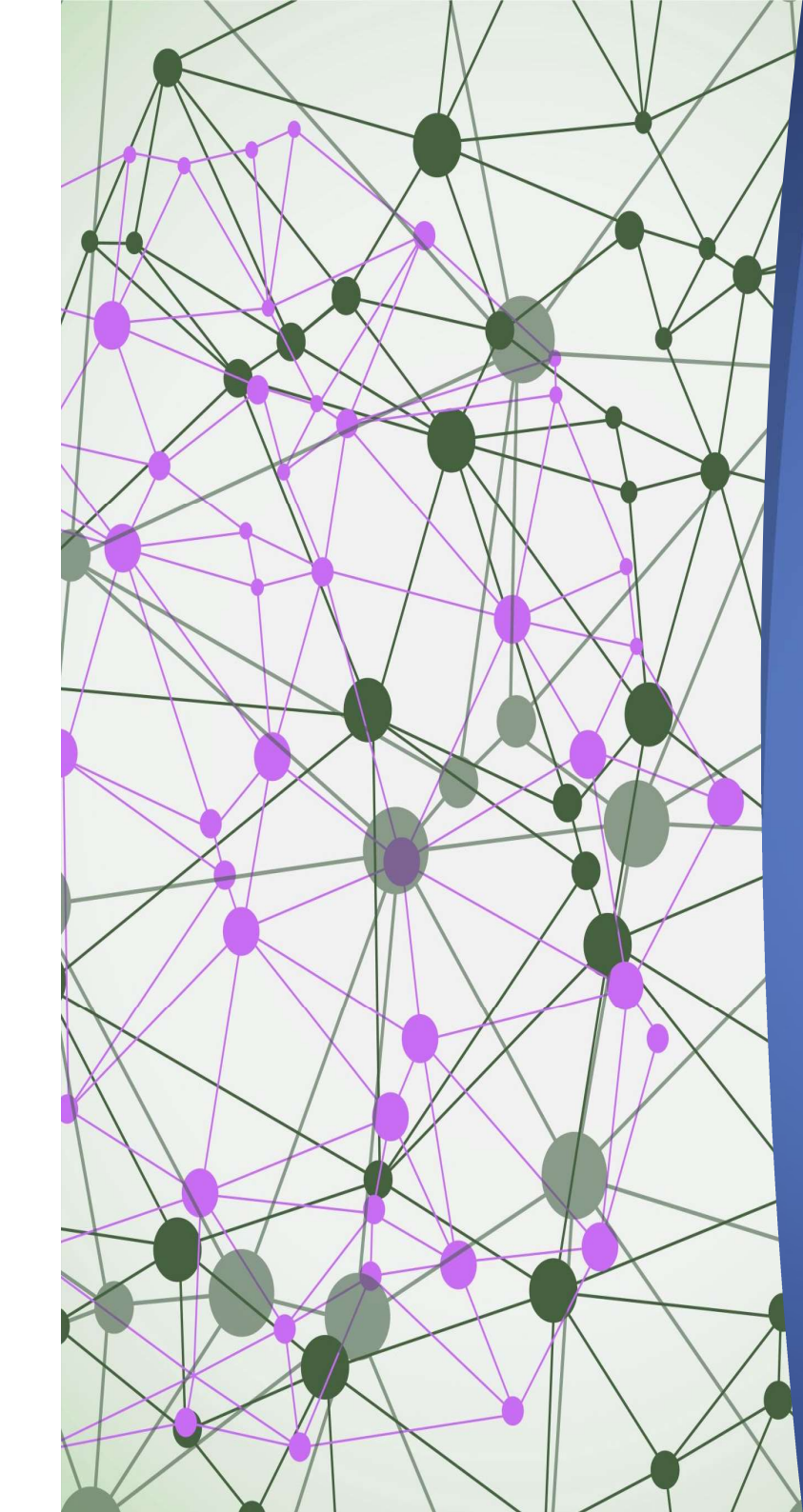




JavaScript

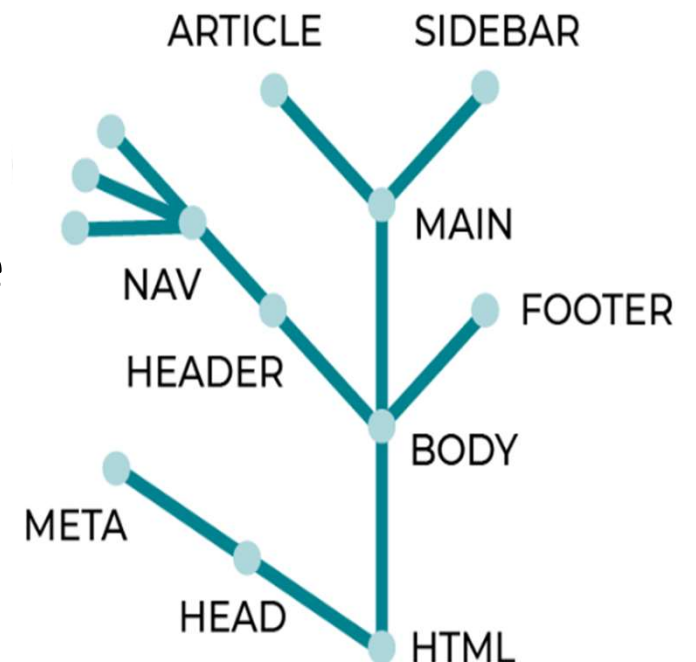
Manipuler les éléments d'une
page html avec DOM

Définition

- **DOM** (**D**ocument **O**bject **M**odel) est un **modèle** pour représenter les documents HTML ou XML.
- Les éléments de ce modèle sont structurés de façon hiérarchique semblable à un arbre composé de plusieurs **nœuds** (Nodes en anglais).

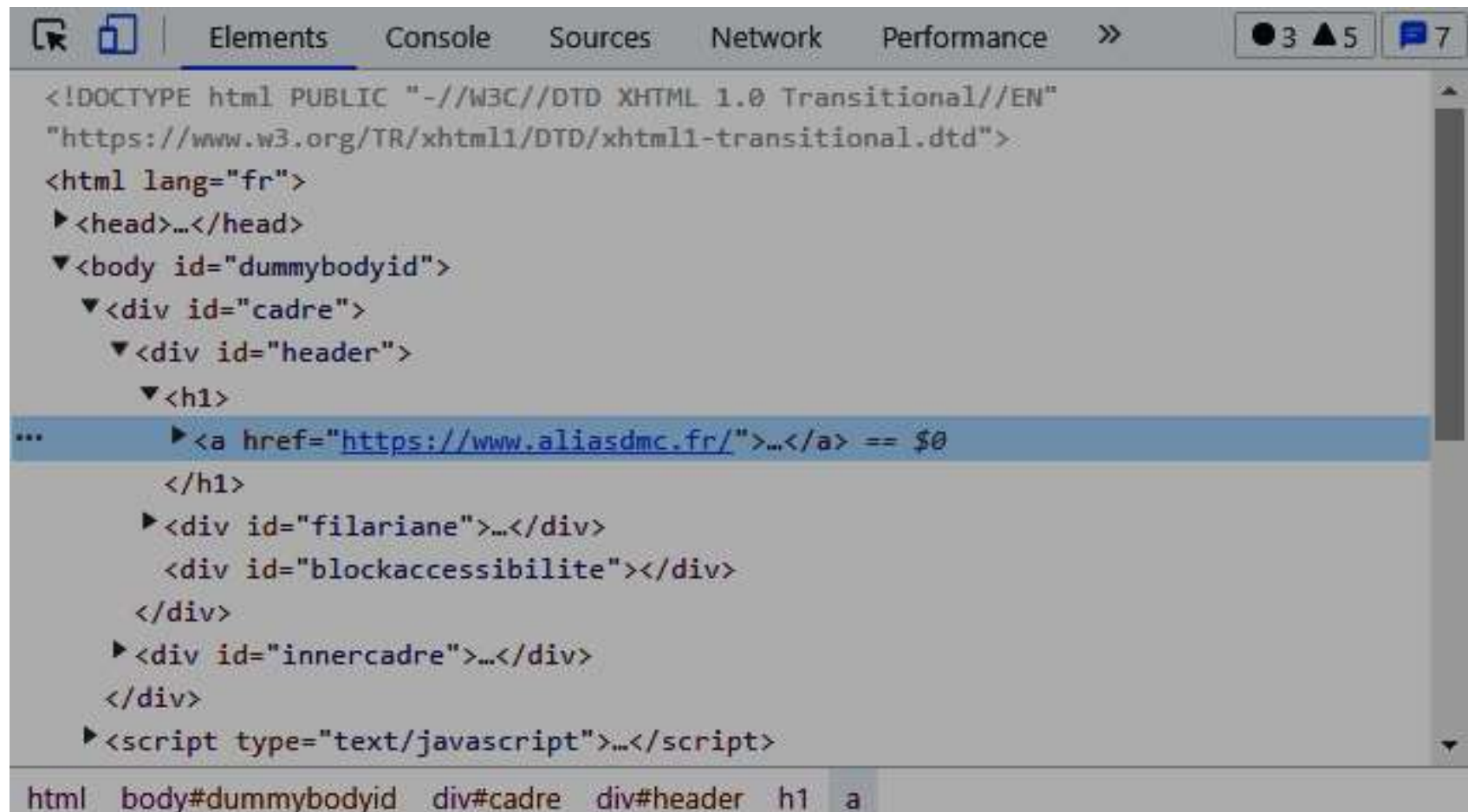
Définition

- Exemple d'arbre **DOM**
 - A partir du nœud racine HTML, on voit que 2 branches se forment :
 - ❑ une première qui va aboutir au nœud HEAD;
 - ❑ une deuxième qui aboutit à un nœud BODY.
 - De nouvelles branches se créent ensuite partir des nœuds HEAD et BODY et etc.



Définition

Une autre représentation du DOM peut être obtenue en inspectant la page HTML.



```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "https://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html lang="fr">
  <head>...</head>
  <body id="dummybodyid">
    <div id="cadre">
      <div id="header">
        <h1>
          ...
          <a href="https://www.aliasdmc.fr/">...</a> == $0
        </h1>
        <div id="filariane">...</div>
        <div id="blockaccessibilite"></div>
      </div>
      <div id="innercadre">...</div>
    </div>
    <script type="text/javascript">...</script>
  </body>
</html>
```

html body#dummybodyid div#cadre div#header h1 a

Définition

- Dans le DOM :
 - Le document lui-même est un nœud de type: **Document**
 - Tous les éléments HTML sont des nœuds de type: **Element**
 - Tous les attributs HTML sont des nœuds de type: **Attribut**
 - Les textes dans HTML sont des nœuds de type: **Text**
 - Les commentaires sont des nœuds de type: **Comment**
- Chaque **nœud** est un objet possède des **propriétés** et des **méthodes**

L'objet Document

- En JavaScript, L'**objet document** fournit des **propriétés** et des **méthodes** pour accéder à tous les nœuds du document HTML.
- Quelques propriétés de l'objet document :
 - **body** retourne le nœud **body** ;
 - **head** retourne le nœud **head** ;
 - **links** retourne une liste de tous les éléments **a** ou **area** possédant un **href** avec une valeur ;
 - **title** retourne le **titre** du document;
 - **url** renvoie l'**url** du document;
- **Exemple :**

```
//Sélectionne l'élément body et applique une couleur bleu
document.body.style.color = 'blue';

//Modifie le texte de l'élément title
document.title= 'Le Document Object Model';
```

L'objet Document

- Quelques méthodes de l'objet document :
 - `querySelector()`
 - `querySelectorAll()`
 - `getElementById()`
 - `getElementsByClassName()`
 - `getElementsByTagName()`
- La méthode **`querySelector()`** retourne un objet **Element** représentant le premier élément dans le document correspondant au sélecteur CSS passé en argument (ou la valeur null si aucun élément n'est pas trouvé).

L'objet Document

- Exemple :

```
<body>
  <h1 class='bleu'>Titre principal</h1>
  <p id='p1'>Un paragraphe</p>
  <div>
    <p>Un paragraphe dans le div</p>
    <p class='bleu'>Un autre paragraphe dans le div</p>
  </div>
  <p>Un autre paragraphe</p>
</body>
```

```
/*Sélectionne le premier paragraphe du document et change son texte avec la
| *propriété textContent que nous étudierons plus tard dans cette partie*/
document.querySelector('p').textContent = '1er paragraphe du document';

let documentDiv = document.querySelector('div'); //1er div du document
//Sélectionne le premier paragraphe du premier div du document et modifie son texte
documentDiv.querySelector('p').textContent = '1er paragraphe du premier div';

/*Sélectionne le premier paragraphe du document avec un attribut class='bleu'
| *et change sa couleur en bleu avec la propriété style que nous étudierons
| *plus tard dans cette partie*/
document.querySelector('p.bleu').style.color = 'blue';
```


L'objet Document

- La méthode **querySelectorAll()** renvoie un objet **NodeList** (une liste des nœuds) qui corresponde au sélecteur CSS passé en argument.
 - Pour itérer dans cette liste des nœuds **NodeList** et accéder à un élément en particulier, on utilise la méthode **forEach()**.
 - **forEach()** prend une fonction à deux arguments qui représentent :
 - L'élément en cours de traitement dans la NodeList ;
 - L'index de l'élément en cours de traitement dans la NodeList ;

L'objet Document

- Exemple :

```
//Sélectionne tous les paragraphes du premier div
let documentDiv = document.querySelector('div'); //1er div du document
let divParas = documentDiv.querySelectorAll('p');

//Sélectionne tous les paragraphes du document
let documentParas = document.querySelectorAll('p');

/*On utilise forEach() sur notre objet NodeList documentParas pour rajouter du
| *texte dans chaque paragraphe de notre document*/
documentParas.forEach(function(nom, index){
|   nom.textContent += ' (paragraphe n°:' + index + ')';
});
```

L'objet Document

- La méthode **getElementById()** renvoie un objet **Element** qui représente l'élément dont la valeur de l'attribut **id** correspond à la valeur spécifiée en argument.
 - Exemple :

```
//Sélectionne l'élément avec un id = 'p1' et modifie la couleur du texte  
document.getElementById('p1').style.color = 'blue';
```

L'objet Document

- La méthode **getElementsByClassName()** renvoie un objet **HTMLCollection** contenant la liste des éléments possédant un attribut **class** avec la valeur spécifiée en argument.
 - Exemple :

```
//Sélectionne les éléments avec une class = 'bleu'
let bleu = document.getElementsByClassName('bleu');
// "bleu" est un objet HTMLCollection qu'on va manipuler comme un tableau
for(valeur of bleu){
    valeur.style.color = 'blue';
}
```

L'objet Document

- La méthode **getElementsByName()** renvoie un objet **HTMLCollection** contenant la liste des éléments correspondant au nom de balise passé en argument.
 - Exemple :

```
//Sélectionne tous les éléments p du document
let paras = document.getElementsByTagName('p');

// "paras" est un objet de HTMLCollection qu'on va manipuler comme un tableau
for(valeur of paras){
    |   valeur.style.color = 'blue';
}
```

L'objet Document

- la méthode **getElementsByName()** renvoie un objet **NodeList** contenant la liste des éléments portant un attribut **name** avec la valeur spécifiée en argument.
 - Exemple :

```
//Sélectionne tous les éléments avec l'attribut name='para'
let paras = document.getElementsByName('para');

// "paras" est un objet de HTMLCollection qu'on va manipuler comme un tableau
for(valeur of paras){
|   valeur.style.color = 'blue';
}
```


Accéder au contenu des éléments

– Exemple :

```
<body>
  <h1 class='bleu'>Titre principal</h1>
  <p id='p1' style="border: 2pt  red solid; color:  blue;">Un paragraphe</p>
  <div style="border: 2pt  black solid">
    <p name="para">Un paragraphe dans le div</p>
    <p class='bleu'>Un autre paragraphe dans le div</p>
  </div>
  <p id="p2">Un autre paragraphe
    <span style="visibility: hidden;">texte invisible</span>
  </p>
</body>
```

```
//Accède au HTML du div et le modifie
document.querySelector('div').outerHTML +=
  | '<ul><li>Elément n°1</li><li>Elément n°2</li></ul>';

//Accède au contenu HTML interne du 1er paragraphe et le modifie
document.querySelector('p').innerHTML += '<h2>Je suis un titre h2</h2>';
```

Accéder au contenu des éléments

- La propriété **textContent** représente le contenu textuel d'un nœud et de ses descendants.;
- La propriété **innerText** représente le contenu textuel visible sur le document final d'un nœud et de ses descendants.
 - Exemple :

Accéder au contenu des éléments

Exemple :

```
// /*Accède au contenu textuel de l'élément avec un id='p1' et le modifie.  
//  *Les balises HTML vont ici être considérées comme du texte*/  
document.getElementById('p1').textContent += '<span>Texte modifié</span>';  
  
// //Accède au texte visible de l'élément avec l'id = 'p2'  
let texteVisible = document.getElementById('p2').innerText;  
  
// //Accède au texte (visible ou non) de l'élément avec l'id = 'p2'  
let texteEntier = document.getElementById('p2').textContent;  
  
// //Affiche les résultats du dessus après l'élément avec l'id = 'p2'  
document.getElementById('p2').outerHTML +=  
    'Texte visible : ' + texteVisible + '<br>Texte complet : ' + texteEntier;
```

Exercices

- **Exercice 1**
- En utilisant le code HTML précédent, reproduisez le code HTML de l'image ci-contre en utilisant JavaScript

Titre principal

Je suis un titre h2

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

- Élément n°1
- Élément n°2
- Élément n°3

Un autre paragraphe + Texte de div :

Un paragraphe dans le div

Un autre paragraphe dans le div

Elément n°1Elément n°2Elément n°3

Navigation dans le DOM

1- Accéder aux enfants d'un nœud

- La propriété **childNodes** renvoie une liste des nœuds enfants de l'élément donné.
- la propriété **children** renvoie les nœuds enfants de type **élément** de l'élément donné.
- **Exemple :**

Navigation dans le DOM

1- Accéder aux enfants d'un nœud

```
let p1 = document.getElementById('p1');

//On accède à tous les noeuds enfants de p1. childNodes renvoie une NodeList
let enfantsP1 = p1.childNodes;

/*On peut ensuite utiliser une boucle forEach() pour tous les manipuler ou
*un indice comme pour les tableaux pour manipuler un noeud enfant en particulier */
enfantsP1[0].textContent = "- " + enfantsP1[0].textContent + "- "
enfantsP1[1].style.fontWeight = 'bold';

//On accède aux noeuds enfants éléments seulement de p1.children renvoie une HTMLCollection
let enfantsEltP1 = p1.children;

//On peut ensuite accéder aux différents enfants comme on le ferait avec un tableau
enfantsEltP1[0].style.textDecoration = 'underline';
```

Navigation dans le DOM

2- Accéder au parent d'un nœud

- La propriété **parentNode** renvoie le parent du nœud spécifié dans le DOM quel que soit son type (élément ou document) ou **null** si le nœud ne possède pas de parent.
- la propriété **parentElement** ne renvoie le parent d'un nœud que s'il s'agit d'un nœud de type **élément** ou **null** sinon.

```
let p2 = document.getElementById('p2');  
p2.parentElement.style.backgroundColor = 'green';  
p2.parentNode.style.borderColor = 'blue';
```

Navigation dans le DOM

- 3- Accéder à un nœud enfant à partir d'un nœud parent.
- La propriété **firstChild** renvoie le **premier nœud enfant** direct d'un nœud ou **null** s'il n'en a pas.
 - La propriété **lastChild** renvoie le **dernier nœud enfant** direct d'un nœud ou **null** s'il n'en a pas.
 - les propriétés **firstElementChild** et **lastElementChild** renvoient respectivement le **premier** et le **dernier nœud enfant** de type élément d'un nœud.

Navigation dans le DOM

3- Accéder à un nœud enfant à partir d'un nœud parent.

```
//On accède au premier noeud enfant de body
let bodyFirstChild = document.body.firstChild;

//On accède au dernier noeud enfant de body
let bodyLastChild = document.body.lastChild;

//On accède au premier noeud enfant élément de body
let bodyFirstChildElement = document.body.firstChildElement;

//On accède au dernier noeud enfant élément de body
let bodyLastElementChild = document.body.lastElementChild;

console.log('Premier enfant de body : ' + bodyFirstChild.textContent + " -");
console.log('Premier enfant élément de body : ' + bodyFirstChildElement.outerHTML);
console.log('Dernier enfant de body : ' + bodyLastChild.textContent + " -");
console.log('Dernier enfant élément de body : ' + bodyLastElementChild.outerHTML);
```

Navigation dans le DOM

4- Accéder au nœud précédent ou suivant un nœud.

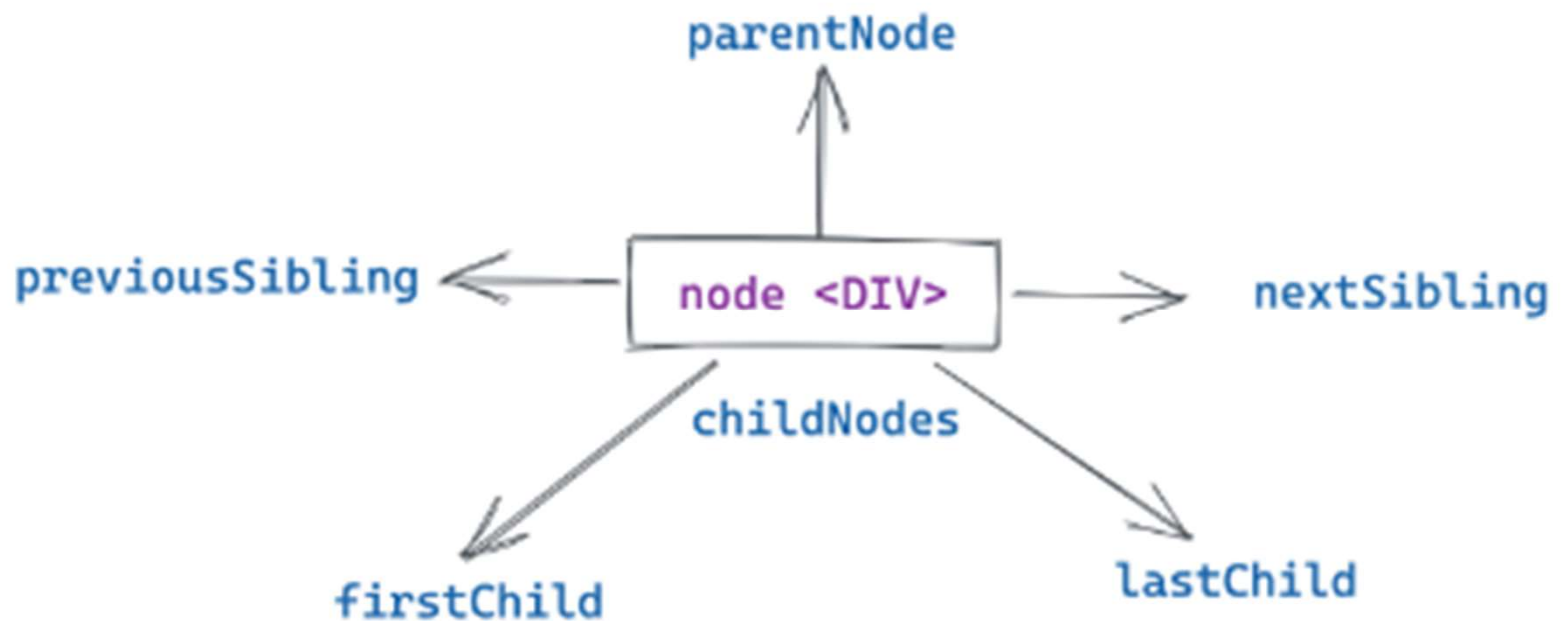
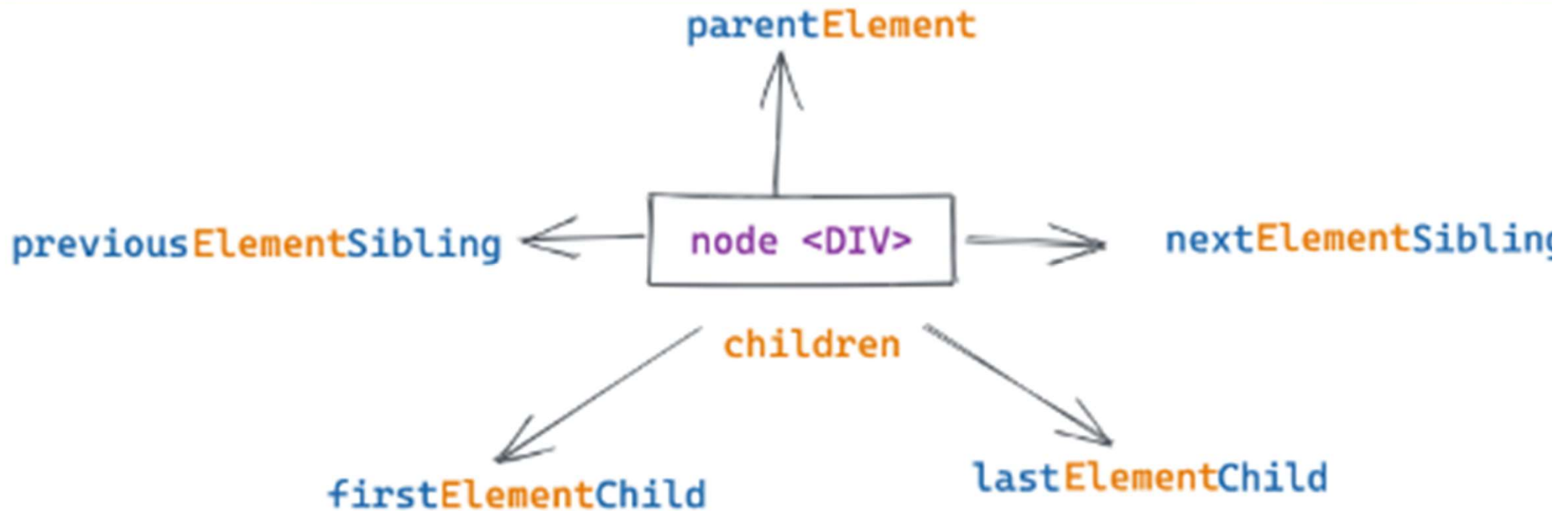
- La propriété **previousSibling** renvoie le nœud précédent un nœud(les nœuds de même niveau) ou **null** si le nœud en question est le premier.
- La propriété **nextSibling** renvoie le nœud suivant un nœud (les nœuds de même niveau) ou **null** si le nœud en question est le dernier.
- les propriétés **previousElementSibling** et **nextElementSibling** accèdent au nœud **élément** précédent ou suivant un nœud.

Navigation dans le DOM

Exemple :

```
let p1 = document.getElementById('p1');  
  
console.log("- " + p1.previousSibling.textContent + " -");  
console.log("- " + p1.nextSibling.textContent + " -");  
  
p1.previousElementSibling.style.color = 'blue';  
p1.nextElementSibling.style.backgroundColor = 'green';
```


Navigation dans le DOM



Navigation dans le DOM

- 5- Accéder au nom d'un nœud, son type ou son contenu.
- La propriété **nodeName** retourne une chaîne de caractères contenant le nom du nœud (nom de la balise dans le cas d'un nœud de type Element ou #text dans le cas d'un nœud de type Text) ;
 - La propriété **nodeValue** renvoie ou permet de définir la valeur du nœud Text;
 - La propriété **nodeType** renvoie un entier qui représente le type du nœud (1 - Nœud élément, 2 - Nœud attribut, 3 - Nœud texte, 8 - Nœud commentaire, 9 - Nœud document).

Navigation dans le DOM

Exemple :

```
<h1>Titre principal</h1>
<p id='p1'>Un paragraphe <span>avec un span</span></p>
<div>
  <p id='p2'>Un paragraphe dans le div</p>
  <p>Un autre paragraphe dans le div</p>
</div>
<p>Un autre paragraphe</p>
```

```
Nom noeud p1 : P
Valeur noeud p1 : null
Type noeud p1 : 1
```

```
let p1 = document.getElementById('p1');
```

```
Nom p1PreviousSibling : #text
```

```
alert(
  'Nom noeud p1 : ' + p1.nodeName +
  '\nValeur noeud p1 : ' + p1.nodeValue +
  '\nType noeud p1 : ' + p1.nodeType +
  '\n\nNom p1PreviousSibling : ' + p1.previousSibling.nodeName +
  '\n\nNom p1PreviousElementSibling : ' + p1.previousElementSibling.nodeName +
  '\nValeur du premier noeud enfant de p1 : ' + p1.firstChild.nodeValue
);
```

```
Nom p1PreviousElementSibling : H1
```

```
Valeur du premier noeud enfant de p1 : Un paragraphe
```

Manipuler les éléments HTML

1- Créer un nœud.

– Nous utilisons les méthodes de l'objet document :

- createElement(' élément ')
- createTextNode(' le texte ')

```
let newP = document.createElement('p');  
newP.textContent = 'Paragraphe créé et inséré grâce au JavaScript';  
  
let newTexte = document.createTextNode('Texte écrit en JavaScript');
```

Manipuler les éléments HTML

2- Insérer un nœud dans le DOM.

- la méthode **prepend()** insère un nœud en tant que premier enfant d'un autre nœud.
- Les méthodes **appendChild()** et **append()** insèrent un nœud en tant que dernier enfant d'un autre nœud.

Manipuler les éléments HTML

- Exemple :

```
let newP = document.createElement('p');
newP.textContent = 'Paragraphe créé et inséré grâce au JavaScript';
let newTexte = document.createTextNode('Texte écrit en JavaScript');

let b = document.body;
//Ajoute le paragraphe créé comme premier enfant de l'élément body
b.prepend(newP);
//Ajoute le texte créé comme dernier enfant de l'élément body
b.append(newTexte);
```


Manipuler les éléments HTML

2- Insérer un nœud dans le DOM.

- Les différences entre **append()** et **appendChild()** sont :
 - La méthode **append()** permet également d'ajouter une chaîne de caractères tandis que **appendChild()** n'accepte que des objets de type Node ;
 - La méthode **append()** peut ajouter plusieurs nœuds et chaînes de caractères au contraire de **appendChild()** qui ne peut ajouter qu'un nœud à la fois ;
 - La méthode **append()** n'a pas de valeur de retour, tandis que **appendChild()** retourne l'objet ajouté.

Manipuler les éléments HTML

Exemple :

```
let newP = document.createElement('p');
newP.textContent = 'Paragraphe créé et inséré grâce au JavaScript';

let newTexte = document.createTextNode('Texte inséré avec appendChild()');

let b = document.body;
//Ceci fonctionne
b.append(newP, 'Texte inséré avec append()');

//Ceci fonctionne
b.appendChild(newTexte);

//Ceci ne fonctionne pas : b.appendChild('Texte écrit en JavaScript');
//Ceci ne fonctionne pas non plus : b.appendChild(newP, newTexte);
```

Manipuler les éléments HTML

2- Insérer un nœud dans le DOM.

- la méthode **insertBefore(arg1, arg2)** insère le nœud **arg1** en tant qu'enfant d'un nœud parent juste avant le nœud enfant **arg2**.

Manipuler les éléments HTML

Exemple :

```
<h1>Titre principal</h1>
<p id='p1'>Un paragraphe <span>avec un span</span></p>
<div>
  <p id='p2'>Un paragraphe dans le div</p>
  <p>Un autre paragraphe dans le div</p>
</div>
<p>Un autre paragraphe</p>
```

```
let newP = document.createElement('p');
newP.textContent = 'Paragraphe créé et inséré grâce au JavaScript';

let b = document.body;
let p1 = document.getElementById('p1');

b.insertBefore(newP, p1);
```

```
<h1>Titre principal</h1>
<p>Paragraphe créé et inséré grâce au JavaScript</p>
▶ <p id="p1">...</p>
```

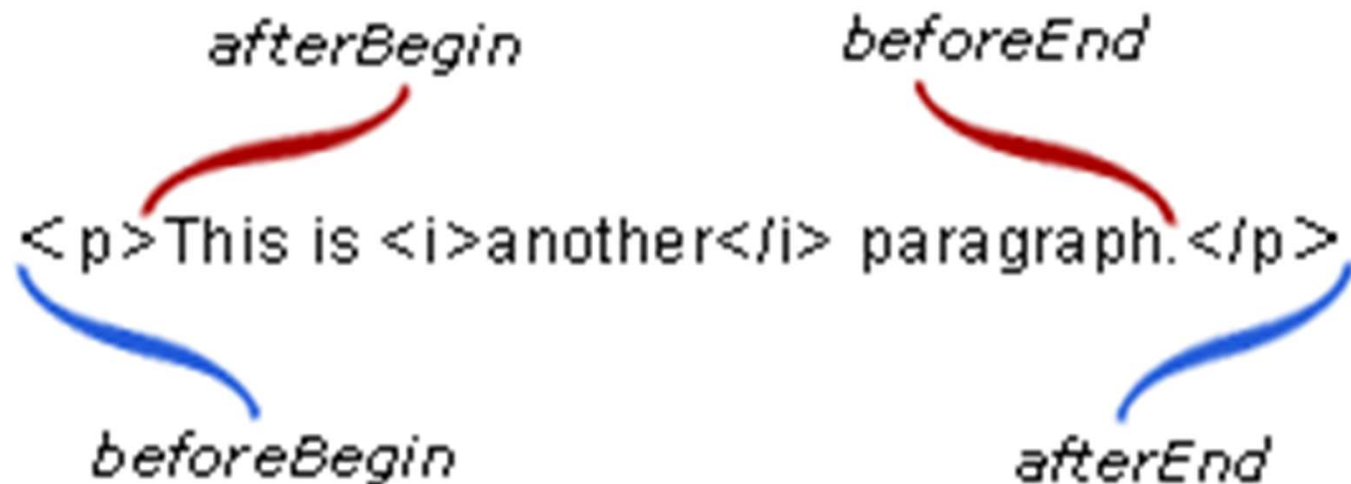
Manipuler les éléments HTML

2- Insérer un nœud dans le DOM.

- La méthode **insertAdjacentElement**(position, élément) insère un nœud **élément** à une position donnée.
- La méthode **insertAdjacentText**(position, texte) insère un nœud **texte** à une position donnée.
- La méthode **insertAdjacentHTML**(position, chaine) analyse une chaîne de caractères en tant que code HTML et insère les nœuds créés avec le balisage à une position spécifiée.

Manipuler les éléments HTML

- Les positions où on souhaite insérer des nœuds sont :
 - **beforebegin** : Insère le ou les nœuds avant l'élément ;
 - **afterbegin** : Insère le ou les nœuds avant le premier enfant de l'élément ;
 - **beforeend** : Insère le ou les nœuds après le dernier enfant de l'élément ;
 - **afterend** : Insère le ou les nœuds après l'élément.



Manipuler les éléments HTML

Exemple :

```
<h1>Titre principal</h1>
<p id='p1'>Un paragraphe <span>avec un span</span></p>
<div>
  <p id='p2'>Un paragraphe dans le div</p>
  <p>Un autre paragraphe dans le div</p>
</div>
<p>Un autre paragraphe</p>
```

```
<h1>Titre principal</h1>
<p id="p1"> == $0
  "Texte ajouté dans P1, "
  "Un paragraphe "
  <span>avec un span</span>
  <strong> et du texte important</strong>
</p>
<p>Paragraphe créé et inséré grâce au JavaScript</p>
<div>...</div>
```

```
let newP = document.createElement('p');
newP.textContent = 'Paragraphe créé et inséré grâce au JavaScript';
let htmlContent = '<strong> et du texte important</strong>';
let p1 = document.getElementById('p1');
//Ajoute un paragraphe après p1
p1.insertAdjacentElement('afterend', newP);
//Ajoute le contenu de htmlContent avant la balise fermante de p1
p1.insertAdjacentHTML('beforeend', htmlContent);
//Ajoute du texte après la balise ouvrante de p2
p1.insertAdjacentText('afterbegin', 'Texte ajouté dans P1, ');
```

Manipuler les éléments HTML

3- Déplacer un nœud dans le DOM.

- La méthode **insertBefore**(élément1, élément2) permet de placer l'**élément1** avant l'**élément2** dans le DOM
- Exemple :

```
let div = document.querySelector('div');  
let p2 = document.getElementById('p2');  
let p3 = div.lastElementChild; //On accède au dernier paragraphe  
  
//On déplace p3 juste avant p2 dans le DOM  
div.insertBefore(p3, p2);
```

Manipuler les éléments HTML

4- Cloner et remplacer un nœud dans le DOM.

- La méthode **cloneNode**(booléen) renvoie une copie du nœud sur lequel elle a été appelée.
 - Si la valeur passée en argument est **true**, les enfants du nœud seront également clonés. Si on lui passe **false** seul le nœud spécifié sera cloné.
- la méthode **replaceChild**(arg1, arg2) remplace le nœud arg2 par le nœud arg1.
- **replaceWith**

Manipuler les éléments HTML

- Exemple :

```
<h1>Titre principal</h1>
<p id='p1'>Un paragraphe <span>avec un span</span></p>
<div>
  <p id='p2'>Un paragraphe dans le div</p>
  <p>Un autre paragraphe dans le div</p>
</div>
<p>Un autre paragraphe</p>
```

```
<div></div>
<p id="p1">...</p>
<div>...</div>
<h1>Titre principal</h1>
```

```
let p1 = document.getElementById('p1');
let div = document.querySelector('div');
let h1 = document.querySelector('h1');

//On clone div et on insère le clone avant p1
let cloneDiv = div.cloneNode(false);
p1.insertAdjacentElement('beforebegin', cloneDiv);

//On remplace le paragraphe qui suit div par h1
document.body.replaceChild(h1, div.nextElementSibling);
```

Manipuler les éléments HTML

5- Supprimer un nœud du DOM.

- la méthode **removeChild()** supprime un nœud enfant passé en argument d'un certain nœud parent de l'arborescence du DOM et retourner le nœud retiré.
- la méthode **remove()** permet tout simplement de retirer un nœud du DOM

Manipuler les éléments HTML

- Exemple :

```
<h1>Titre principal</h1>
<p id='p1'>Un paragraphe <span>avec un span</span></p>
<div>
  <p id='p2'>Un paragraphe dans le div</p>
  <p>Un autre paragraphe dans le div</p>
</div>
<p>Un autre paragraphe</p>
```

```
let p1 = document.getElementById('p1');
let p2 = document.getElementById('p2');
```

```
<h1>Titre principal</h1>
<div>
  <p>Un autre paragraphe dans le div</p>
</div>
<p>Un autre paragraphe</p>
```

```
//Supprime p1 du DOM et renvoie le noeud supprimé
let eltDel=document.body.removeChild(p1)
console.log('Noeud supprimé du DOM : ' + eltDel.outerHTML);
//Supprime p2 du DOM
p2.remove()
```

Noeud supprimé du DOM : <p id="p1">Un paragraphe avec un span</p>

Exercices

- **Exercice 2 :**

Créez la structure HTML ci-dessous à l'aide de JavaScript :

```
<div id='div1'>
  <p> Langages de programmation :</p>
  <ul>
    <li>JavaScript</li>
    <li>Python</li>
    <li>PHP</li>
  </ul>
</div>
```

Exercices

- **Exercice 3 :**

Soit la structure HTML suivante :

```
<h1>Titre principal</h1>
<div>
  <p class='blue'>Paragraphe n°:1</p>
  <p id='p2'>Paragraphe n°:2</p>
  <p>Paragraphe n°:3</p>
  <p>Paragrahe sans attribut</p>
</div>
<p>Paragraphe n°:4</p>
```

En utilisant JavaScript,

1. Copiez le paragraphe n°: 4 avant la fin de div
2. Déplacez le paragraphe n°: 2 après le titre principale
3. Supprimez le paragraphe n°: 3
4. Remplacez le paragraphe sans attribut par le paragraphe n°: 1

Manipuler les attributs des éléments HTML

1- Tester la présence d'attributs.

- La méthode **hasAttribute(arg)** teste la présence d'un attribut dans un élément.
 - Cette méthode renvoie la valeur **true** si l'élément possède l'attribut **arg** ou **false** sinon.
- La méthode **hasAttributes()** vérifie si un élément possède des attributs ou pas.
 - Cette méthode retourne **true** si l'élément possède au moins un attribut ou **false** si l'élément ne possède pas d'attribut.

Manipuler les attributs des éléments HTML

Exemple :

```
<p id='p1' class='blue'>Un paragraphe</p>  
<p id='vide'></p>
```

```
let p1 = document.querySelector('p');  
let vide = document.getElementById('vide');  
  
//Si p1 possède des attributs, hasAttributes() renvoie true  
if (p1.hasAttributes())  
  vide.textContent = 'p1 possède des attributs';  
  
//Si p1 possède un attribut id, hasAttribute() renvoie true  
if (p1.hasAttribute('id'))  
  vide.textContent += ' dont un attribut id';
```

```
<p id="p1" class="blue">Un paragraphe</p>
```

```
<p id="vide">p1 possède des attributs dont un attribut id</p>
```

Manipuler les attributs des éléments HTML

2- Récupérer la valeur ou le nom d'un attribut.

- La propriété **attributes** renvoie la liste des attributs d'un élément spécifié.
 - On peut récupérer les noms et valeurs de chaque attribut en utilisant la boucle for.

Manipuler les attributs des éléments HTML

- Exemple :

```
<p id='p1' class='blue'>Un paragraphe</p>
<p id='vide'></p>
```

```
<p id="p1" class="blue">Un paragraphe</p>
<p id="vide">
  "Liste des attributs de p1 : "
  <br>
  "id = p1"
  <br>
  "class = blue"
  <br>
</p>
```

```
let p1 = document.querySelector('p');
let vide = document.getElementById('vide');

if (p1.hasAttributes()) {
  let attP1 = p1.attributes; // Liste des attributs de p1
  vide.innerHTML = 'Liste des attributs de p1 : <br>'
  for (let att of attP1)
    vide.innerHTML += att.name + ' = ' + att.value + '<br>';
}
```


Manipuler les attributs des éléments HTML

2- Récupérer la valeur ou le nom d'un attribut.

- la méthode **getAttributeNames()** qui renvoie les noms des attributs d'un élément sous forme de tableau
- la méthode **getAttribute(arg)** renvoie la valeur de l'attribut **arg** si celui-ci existe ou null le cas contraire.

`classList.add .remove .replace`

Manipuler les attributs des éléments HTML

- Exemple :

```
<p id='p1' class='blue'>Un paragraphe</p>
<p id='vide'></p>
```

```
<p id="p1" class="blue">Un paragraphe</p>
<p id="vide">
  "Attributs de p1 : "
  <br>
  "id = p1"
  <br>
  "class = blue"
  <br>
</p>
```

```
let p1 = document.querySelector('p');
let vide = document.getElementById('vide');

//Si p1 possède des attributs...
if(p1.hasAttributes()){
  //On récupère leurs noms dans un tableau
  let attP1 = p1.getAttributeNames();
  vide.innerHTML = 'Attributs de p1 : <br>';

  //Pour chaque élément du tableau...
  for(nom of attP1){
    //On récupère la valeur associée au nom de l'attribut
    let valeur = p1.getAttribute(nom);
    //On affiche le nom et la valeur de l'attribut
    vide.innerHTML += nom + ' = ' + valeur + '<br>';
  }
}
```

Manipuler les attributs des éléments HTML

3- Définir la valeur ou le nom d'un attribut.

- la méthode **setAttribute**(arg1, arg2) permet d'ajouter un nouvel attribut ou changer la valeur d'un attribut existant pour un élément.
 - Arg1 est le **nom** de l'attribut à ajouter ou à modifier.
 - Arg2 est la **valeur** de l'attribut à ajouter ou à modifier.
- les propriétés **className** et **id** permettent respectivement d'obtenir ou de définir la valeur d'un attribut **class** et **id**.

Manipuler les attributs des éléments HTML

- Exemple :

```
<p id='p1' class='blue'>Un paragraphe</p>
<p id='vide'></p>
```

```
let vide = document.getElementById('vide');
vide.setAttribute('class', 'blue');
vide.innerHTML += 'class : ' + vide.className + '<br>id : ' + vide.id;
```

```
<p id="p1" class="blue">Un paragraphe</p>
<p id="vide" class="blue">
  "class : blue"
  <br>
  "id : vide"
</p>
```

Manipuler les attributs des éléments HTML

4- Supprimer un attribut.

- la méthode **removeAttribute(arg)** supprime l'attribut passé en argument d'un élément.
- Exemple :

```
<p id='p1' class='blue'>Un paragraphe</p>  
<p id='vide'></p>
```

```
let p1 = document.querySelector('p');  
p1.removeAttribute('class');
```

```
<p id="p1">Un paragraphe</p>  
<p id="vide"></p>
```

Modifier les styles d'un élément HTML

- La propriété **style** permet de définir les styles d'un élément

```
<p id='p1' class='blue'>Un paragraphe</p>
<p id='vide'></p>
```

```
<p id="p1" class="blue" style="color: green; font-size: 28px;
background-color: red; border: 5px solid blue;">Un paragraphe
</p>
<p id="vide"></p>
```

```
<p id="p1" class="blue" style="border:5px solid blue;">Un paragraphe</p>
<p id="vide"></p>
```

```
let p1 = document.querySelector('p');
p1.style.color = 'green';
p1.style.fontSize = '28px';
p1.style.backgroundColor = 'red';
p1.style.border = '5px solid blue';
```

```
let p1 = document.querySelector('p');
p1.style.color = 'green';
p1.style.fontSize = '28px';
p1.style.backgroundColor = 'red';
p1.style.border = '5px solid blue';
// setAttribute remplace l'attribut style
p1.setAttribute('style', 'border:5px solid blue;');
```


Exercices

- **Exercice 4 :**

On considère le code HTML ci-dessous :

```
<p class="vert">Paragraphe n°: 1</p>  
<p>Paragraphe n°: 2</p>  
<p class="vert">Paragraphe n°: 3</p>  
<p id="p4">Paragraphe n°: 4</p>
```

- 1- Ecrire un code JavaScript qui permet d'ajouter la balise **<p> Paragraphe numéro 5 </p>** entre le paragraphe 2 et le paragraphe 3
- 2- Ajouter une bordure et une couleur de l'arrière plan au paragraphe numéro 2 avec deux manières différentes
- 3- Colorier les paragraphes possédant l'attribut class='vert' en vert
- 4- Ajouter au paragraphe 4 l'attribut class= 'rouge'
- 5- Supprimer l'attribut id= 'p4' du paragraphe 4